Univerzitet u Zenici
Politehnički fakultet

BILJEŠKE S PREDAVANJA · SEDMICA 1

Od virtualnih mašina do kontejnera

Virtualizacija i kontejnerizacija: izgradnja prototipa izvršnog okruženja za kontejnere polazeći od sistemskog poziva `chroot`.

PREDMET	Računarstvo u oblaku
STUDIJ	II ciklus studija, III semestar, akademska godina 2026/27
NASTAVNIK	V. prof. dr. Đulaga Hadžić
ASISTENT	ass. Mr. Adin Jahić
TERMIN	petak, 16.10.2026, 17:15–20:15, Kabinet 3204
TEMA	Virtualizacija i kontejnerizacija

OKVIR NASTAVNE JEDINICE

Ciljevi i način rada

Nastavna jedinica razmatra kontejner kao kombinaciju triju mehanizama jezgra operativnog sistema: promjene korijena fajl sistema, izolacije pogleda na resurse (*namespaces*) i ograničavanja potrošnje (*cgroups*). Gradivo se izlaže induktivno: počinje se najjednostavnijim mogućim izvršnim okruženjem, a svaki naredni isječak koda dodaje jednu funkcionalnost i otklanja jedno uočeno ograničenje prethodnog.

Nakon nastavne jedinice student je u stanju:

- objasniti razliku između virtualne mašine i kontejnera na nivou jezgra operativnog sistema,
- opisati ulogu poziva `chdir` i `chroot` te način na koji jezgro razrješava putanje,
- objasniti zašto dinamički povezan program ne radi u praznom korijenu (format ELF),
- izgraditi minimalno izvršno okruženje koje pokreće fajl sistem izvučen iz OCI slike,
- razlikovati doprinos *namespaces* i *cgroups* te pokazati ga na radnom primjeru.

Preduslovi za izvođenje isječaka

- Linux ili WSL2 (Ubuntu) s pravom `sudo`; potreban je pristup internetu za instalaciju paketa.
- Docker okruženje, potrebno samo za isječke 9, 10 i 13, koji preuzimaju stvarne OCI slike.
- Svi isječci izvršavaju se redom, u istom direktoriju (`~/kontejneri`). Svaki se oslanja na datoteke nastale u prethodnim, pa se kopiraju i pokreću u navedenom redoslijedu.

```bash
# Isječak 0 · priprema
sudo apt update && sudo apt install -y gcc binutils busybox-static
mkdir -p ~/kontejneri && cd ~/kontejneri
```

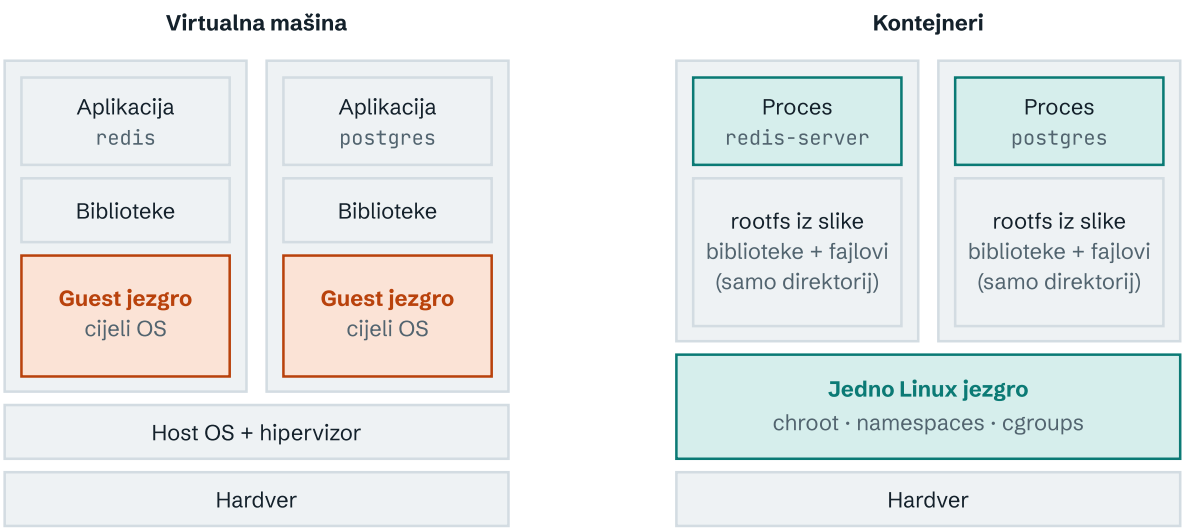
---

1 · POJAM KONTEJNERA

Kontejner nije virtualna mašina

U svakodnevnoj praksi kontejner se doživljava kao „kutija“ pokrenuta naredbom `docker run`, a u opisima se često svodi na tvrdnju da je „lakši oblik virtualne mašine“. Takav opis je operativno koristan, ali ne objašnjava mehanizam. Zbog toga se u nastavku apstrakcija postepeno uklanja, sve do jednog sistemskog poziva.

Virtualna mašina uključuje vlastito jezgro i cjelokupni operativni sistem, a hipervizor emulira ili dijeli hardver. Kontejner nema vlastito jezgro: svi kontejneri na računaru koriste jedno jezgro domaćina, a svaki je običan proces kojem je ograničen pogled na fajl sistem i druge resurse.



Slika 1. Virtualna mašina nosi cijeli operativni sistem po aplikaciji. Kontejner nema vlastito jezgro: to je proces čiji je pogled na fajl sistem ograničen na jedan direktorij.

> **Pokrenuti kontejner je proces. Ne postoji ni virtualna mašina ni hipervizor.**

Iz toga slijedi da kontejner ne uvodi poseban trošak pri izvršavanju, jer nema posrednika između procesa i jezgra. Primjetno sporije pokretanje obično potiče od veličine slike, njenog preuzimanja s mreže ili dijeljenja datoteka s virtualnom mašinom na sistemima Mac i Windows, a ne od samog kontejnera.

RADNA TEZA

Kontejner je „chroot na steroidima“ (formulacija iz ranih godina Dockera). Pojačanje čine *namespaces* i *cgroups*. Zato se u nastavku najprije gradi izvršno okruženje zasnovano samo na `chroot` -u, a zatim se dopunjuje.

---

2 · IZVRŠNO OKRUŽENJE KONTEJNERA

Izvršno okruženje pokreće sliku koja miruje

OCI slika je zapis na disku, po svojstvima sličan izvršnom fajlu. Kada se pokrene, postaje kontejner. Program koji obavlja to pokretanje naziva se *izvršno okruženje kontejnera* (engl. *container runtime*). Postojeća izvršna okruženja se međusobno razlikuju po složenosti, a ovdje se gradi onaj na sasvim jednostavnom kraju spektra.

IZVRŠNO OKRUŽENJE	NIVO	KARAKTERISTIKE
`runc`	nizak	Najmanji i najrašireniji korišten. Sam ne preuzima slike s registra.
`containerd`	srednji	Preuzima i pokreće OCI/Docker slike, a izvršavanje delegira runc-u.
Kata, Firecracker	visok	Izvršavaju kontejnere unutar mikro virtualnih mašina. Koriste infrastrukturu virtualizacije, pa nisu izvršna okruženja u užem smislu.
`bocker`	eksperimentalni	Većina Docker CLI-ja u oko 200 linija bash-a. Kod je vrlo gust i teško čitljiv.
`chrun` (ovdje)	edukativni	Samo `chroot` i raspakovana slika, kasnije proširen namespaces mehanizmom. Nije za produkciju.

---

3 · PORIJEKLO

chdir i chroot: dva pokazivača u Unixu 1979.

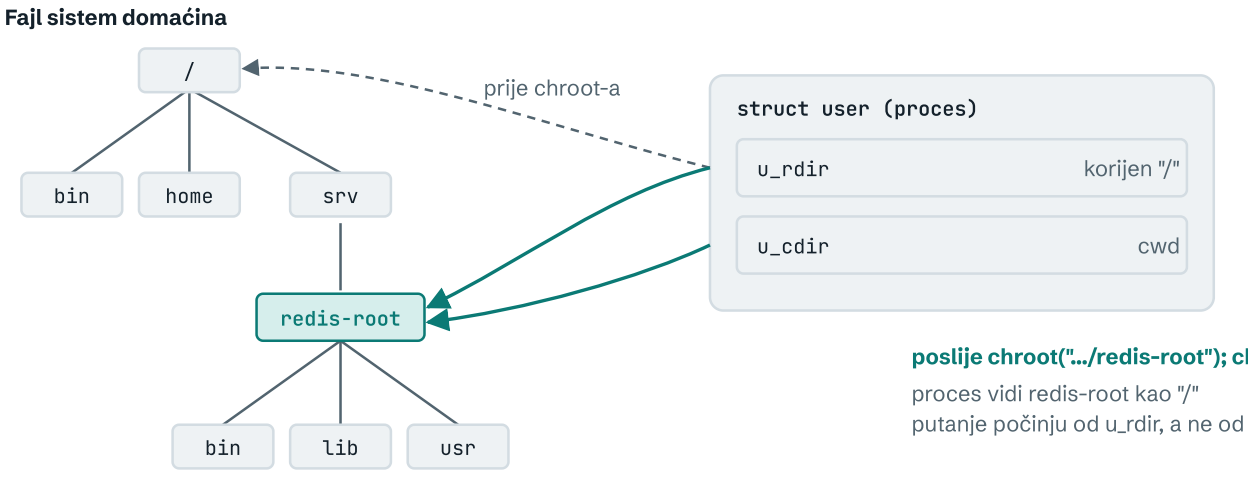
Sistemski poziv `chroot` stariji je od Linuxa. Prva verzija Linuxa (0.01) već ga sadrži u jednostavnom obliku, a on potiče iz Unixa, gdje su `chdir` i `chroot` dodani zajedno 7. maja 1979. godine. Ta verzija je izuzetno kratka i zato pogodna za razumijevanje principa.

Jezgro za svaki proces čuva strukturu s dva pokazivača na čvorove fajl sistema. Poziv `chdir` mijenja prvi, a `chroot` drugi:

unix-v7-skica.c — pojednostavljena skica, ne doslovni izvor

```
struct user {                   // podaci o trenutnom procesu
    struct inode *u_cdir;       // trenutni direktorij ("cwd")
    struct inode *u_rdir;       // korijen ("/") ovog procesa
    /* ... */
};

chdir()  { chdirec(&u.u_cdir); }                    // promijeni cwd
chroot() { if (suser()) chdirec(&u.u_rdir); }       // promijeni korijen
(samo root)
```



Slika 2. Proces ima dva pokazivača u stablo fajl sistema. `chroot` pomjera `u_rdir`, `chdir` pomjera `u_cdir`. Svi ostali mehanizmi kontejnerizacije nadograđuju se na ovu osnovu.

Ključan zaključak je da se promjena korijena svodi na izmjenu jedne vrijednosti u opisu procesa: postavlja se drugačija početna tačka za razrješavanje apsolutnih putanja. Sve ostalo u kontejnerima predstavlja dopunu ove ideje.

POZIV	MIJENJA	EFEKAT NA PUTANJE
`chdir(p)`	`u_cdir`	Relativne putanje (`a/b`) razrješavaju se od novog direktorija.
`chroot(p)`	`u_rdir`	Apsolutne putanje (`/etc/x`) razrješavaju se od novog korijena. Ništa izvan njega nije dostupno.

4 · LINUX

Isti pozivi u Linuxu i prvi radni prototip

Linux ova dva poziva nasljeđuje kao `chdir(2)` i `chroot(2)`. Za izgradnju prototipa bitna su tri svojstva:

- `chroot()` zahtijeva privilegiju (`CAP_SYS_CHROOT`), pa se u praksi izvodi kao root ili preko `sudo` .
- `chroot()` **ne mijenja** trenutni direktorij. Ako se izostavi `chdir("/")` , proces zadržava cwd izvan novog korijena, što je poznata sigurnosna slabost.
- Nakon promjene korijena poziva se `execv()` : proces se zamjenjuje novim programom, a korijen se nasljeđuje.

Prvi isječak koristi gotovu komandu `chroot` i statički izgrađen program. Statički izgrađen znači da program ne traži nijednu dijeljenu biblioteku, pa mu je dovoljan prazan korijen s jednim fajlom. Drugi isječak isto ostvaruje vlastitim programom, što je ujedno i cijelo minimalno izvršno okruženje.

bash — Isječak 1 · hello u praznom korijenu

```
cat > hello.c <<'EOF'
#include <stdio.h>
int main(void) { puts("Hello iz kontejnera!"); return 0; }
EOF
gcc -static hello.c -o hello          # -static: bez dijeljenih biblioteka

mkdir -p root1 && cp hello root1/
sudo chroot root1 /hello              # alat iz coreutils-a
```

izlaz

```
Hello iz kontejnera!
```

bash + C — Isječak 2 · chrun_min: chroot + chdir + exec

```
cat > chrun_min.c <<'EOF'
// chrun_min.c: najmanji mogući "runtime": chroot + chdir + exec
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc < 3) {
        fprintf(stderr, "upotreba: %s <root> <program> [args...]\n", argv[0]);
        return 1;
    }
    if (chroot(argv[1]) < 0) { perror("chroot"); return 1; }  // u_rdir =
argv[1]
    if (chdir("/")      < 0) { perror("chdir");  return 1; }  // u_cdir = novi
korijen
    execv(argv[2], &argv[2]);                                 // zamijeni
proces programom
    perror("execv");                                          // ovdje se stiže
samo pri grešci
    return 1;
}
EOF
gcc chrun_min.c -o chrun_min
sudo ./chrun_min root1 /hello         # isti rezultat kao s alatom chroot
```

izlaz

```
Hello iz kontejnera!
```

Program iz drugog isječka čini tačno ono što i komanda `chroot`, uz deset linija C koda. Sistemski pozivi su isti, tako da u pokretanju nema dodatne „magije“.

---

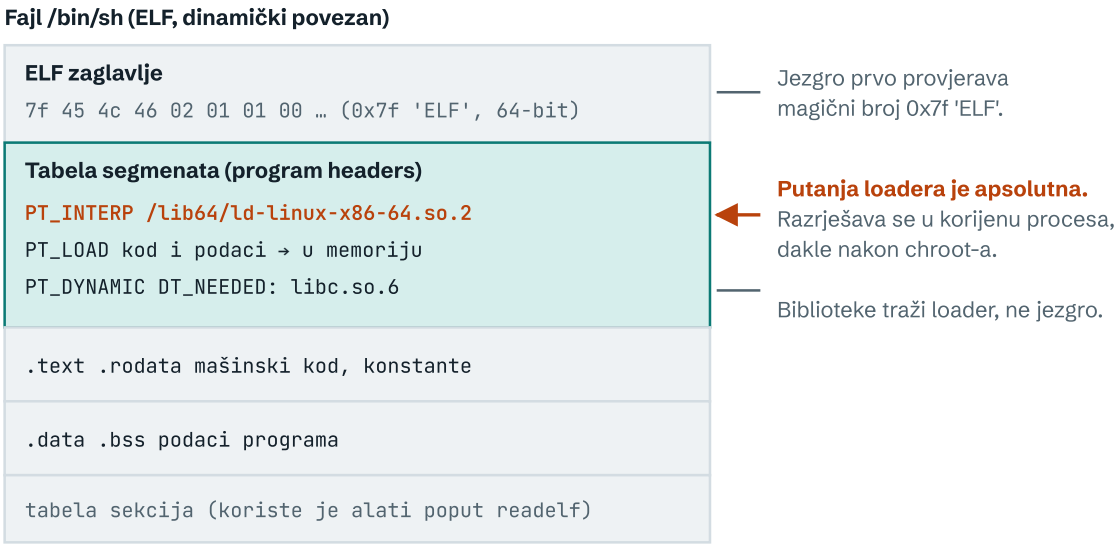
5 · IZVRŠNI FAJL NA LINUXU

Format ELF i zašto program traži biblioteke

Sljedeći korak prototipa je pokretanje ljuske umjesto jednostavnog statičkog programa. Pokušaj tog koraka ne uspijeva, a razlog je u načinu na koji su izvršni fajlovi organizirani. Zbog toga se prije nastavka razmatra format **ELF** (*Executable and Linkable Format*), standardni format izvršnih fajlova i biblioteka na Linuxu.

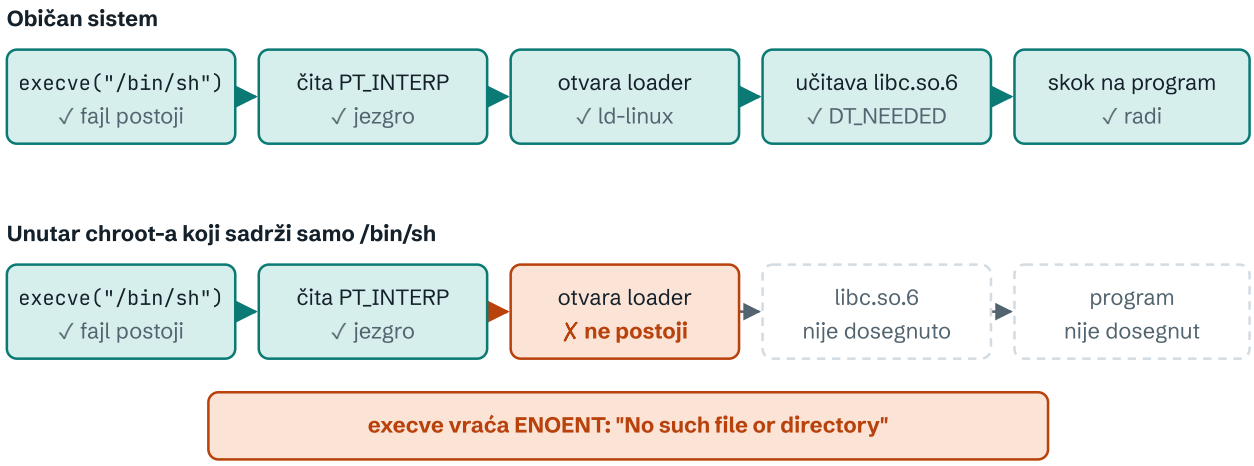
ELF fajl nije samo niz mašinskih instrukcija. Počinje zaglavljem koje ga identifikuje, slijedi tabela zaglavlja programa (*program headers*) koja jezgru opisuje kako proces treba napraviti u memoriji, zatim sekcije s kodom i podacima. Za dinamički povezane programe

tabela segmenata sadrži dva podatka koja su ovdje presudna: putanju **loadera** (`PT_INTERP`) i spisak potrebnih biblioteka (`DT_NEEDED`).



Slika 3. Raspored ELF fajla. Za temu su presudna dva zapisa: `PT_INTERP` (koji program učitava ostale dijelove) i `DT_NEEDED` (koje dijeljene biblioteke ti dijelovi traže).

Pri pozivu `execve` jezgro čita zaglavlje i tabelu segmenata. Ako postoji `PT_INTERP`, otvara navedeni loader (`ld-linux`) i njemu prepušta kontrolu. Loader zatim otvara svaku biblioteku iz `DT_NEEDED` (obično `libc.so.6`), mapira je u memoriju i tek onda skače na ulaznu tačku programa. Sve se otvaranje datoteka odvija po putanjama razriješenim u korijenu procesa.



Slika 4. Poruka o nepostojećem fajlu odnosi se na loader, a ne na ljusku koja je zaista prisutna. Loader i biblioteke moraju postojati na istim putanjama *unutar* novog korijena.

Sadržaj ELF zaglavlja i spisak zavisnosti mogu se pregledati običnim alatima. Sljedeći isječak ispisuje magične bajtove, traženi loader, tražene biblioteke i, naposljetku, koje od njih zaista postoje na disku:

```bash
od -An -tx1 -N16 /bin/sh                          # prvih 16 bajta fajla (ELF zaglavlje)
readelf -l /bin/sh | grep interpreter             # PT_INTERP: koji loader traži
readelf -d /bin/sh | grep NEEDED                  # DT_NEEDED: koje biblioteke traži
ldd /bin/sh                                       # šta od toga zaista postoji na disku
```

Isječak 3 · pregled ELF fajla

```
izlaz (adrese se razlikuju od sistema do sistema)

 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
      [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
 0x…0001 (NEEDED)             Shared library: [libc.so.6]
       linux-vdso.so.1 (0x…7000)
       libc.so.6 ⇒ /lib/x86_64-linux-gnu/libc.so.6 (0x…0000)
       /lib64/ld-linux-x86-64.so.2 (0x…e000)
```

Prvi red potvrđuje magični broj `7f 45 4c 46`, odnosno 0x7f i slova ELF. Iz ispisa se vidi da `/bin/sh` traži loader `/lib64/ld-linux-x86-64.so.2` i biblioteku `libc.so.6`. Slika 3 prikazuje gdje se u fajlu ti podaci nalaze.

---

6 · PROTOTIP KORAK PO KORAK

Od ljuske u praznom korijenu do Redisa iz Docker slike

Svaki od sljedećih koraka dodaje jednu mogućnost. Isječci se pokreću redom, a na grešku koja se pritom pojavi odgovara sljedeći isječak.

6.1 Neuspjeh: ljuska u korijenu bez biblioteka

U korijen se kopira samo `/bin/sh`. Fajl postoji, ali pokretanje završava greškom koja, prema slici 4, zapravo označava izostanak loadera.

```bash
mkdir -p root2/bin && cp /bin/sh root2/bin/    # u korijen se kopira samo ljuska
sudo ./chrun_min root2 /bin/sh -c 'echo ok'
echo "izlazni kod: $?"
```

Isječak 4 · očekivana greška

izlaz

```
execv: No such file or directory
izlazni kod: 1
```

6.2 Popravka (a): kopiranje zavisnosti na iste putanje

Zavisnosti ispisane alatom `ldd` kopiraju se u korijen na identične putanje. Za rekurzivne zavisnosti postoji alat `lddtree`, a ovdje je dovoljna jedna razina.

bash — Isječak 5 · ljuska s bibliotekama

```
for l in $(ldd /bin/sh | grep -o '/[^ ]*'); do
    mkdir -p root2$(dirname $l)       # isti direktorij unutar novog korijena
    cp $l root2$l                     # ista biblioteka na istoj putanji
done
sudo ./chrun_min root2 /bin/sh -c 'echo ok; echo /*'
```

izlaz

```
ok
/bin /lib /lib64
```

Ispis `echo /*` prikazuje sadržaj novog korijena: samo `/bin`, `/lib` i `/lib64`. Ostatak fajl sistema domaćina nije vidljiv.

6.3 Popravka (b): statički povezan program

Drugo rješenje je program koji uopće ne traži biblioteke. Paket `busybox-static` isporučuje u jednom fajlu ljusku i uobičajene alate (`ls`, `cat`, `ps`, `hostname`). Taj korijen koriste i kasniji isječci.

bash — Isječak 6 · statički busybox

```
mkdir -p root3/bin root3/proc && cp /bin/busybox root3/bin/    # statički busybox
sudo ./chrun_min root3 /bin/busybox ls /                       # vidi se samo bin i proc
sudo ./chrun_min root3 /bin/busybox cat /etc/os-release || echo "/etc/os-release domaćina nije vidljiv"
```

izlaz

```
bin
proc
cat: can't open '/etc/os-release': No such file or directory
/etc/os-release domaćina nije vidljiv
```

6.4 „Kontejner u kontejneru“ je grana istog stabla

Kada se i samo izvršno okruženje izgradi statički, može se postaviti u korijen i pokrenuti iznutra. To radi, ali ne predstavlja ugniježđenje u pravom smislu. Postoji jedno stablo fajl sistema, a svaka grana tog stabla je opet stablo. Proces dobiva jednu granu kao svoj korijen, a on od nje može odabrati novu granu za sljedeći proces.

bash — Isječak 7 · chroot unutar chroot-a

```
gcc -static chrun_min.c -o chrun_min_static
mkdir -p root3/inner && cp hello root3/inner/
cp chrun_min_static root3/bin/
sudo ./chrun_min root3 /bin/chrun_min_static /inner /hello    # chroot unutar
chroot-a
```

izlaz

```
Hello iz kontejnera!
```

POSLJEDICA

Alati koji „pokreću kontejnere u kontejnerima“ ne slažu zatvorene kutije jednu u drugu. Oni samo biraju različite grane fajl-sistemskog stabla, a tvrdnja da je sloj već izgrađen (i da se ne mora ponovo graditi) smislena je upravo zato što je sloj samo direktorij.

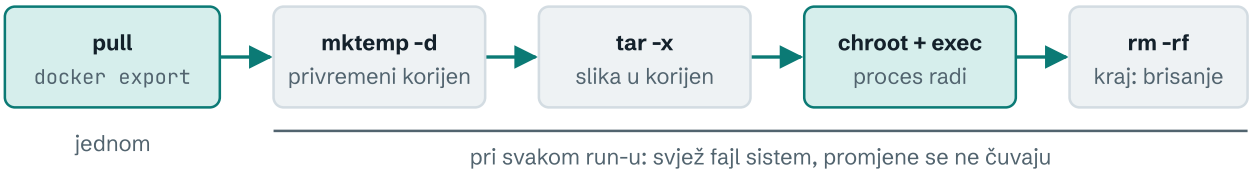
6.5 Fajl sistem iz OCI slike

Ručno kopiranje biblioteka ne skalira. Prirodniji izvor korijena je gotova OCI slika, čiji je fajl sistem već ispravno sastavljen. Slika se sastoji od slojeva (`layer.tar`), od kojih svaki odgovara jednom koraku iz Dockerfile-a. Mogli bi se raspakovati redom jedan preko drugog, ali naredba `docker export` taj posao obavlja i daje jedan tar sa spojenim fajl sistemom.



Slika 5. Prototip pri svakom pokretanju raspakuje cijelu sliku (slika od 4 GB znači 4 GB kopiranja). Prava izvršna okruženja slojeve ostavljaju nepromijenjene i montiraju ih jedan preko drugog, pa se zajednički sloj preuzima i čuva samo jednom.

Cijeli tok pokretanja prikazan je na slici 6, a implementiran je u skripti `chrun.sh`. Skripta ima dvije naredbe: `pull` preuzima sliku i pohranjuje je kao tar, a `run` pravi privremeni korijen, raspakuje sliku, pokreće proces i po završetku briše korijen. Zbog toga je svako pokretanje „svježe“, a promjene unutar kontejnera se ne čuvaju.



Slika 6. Cijelo izvršno okruženje prototipa: preuzimanje, privremeni korijen, raspakivanje, izvršavanje i čišćenje.

bash — Isječak 8 · skripta chrun.sh

```
cat > chrun.sh <<'EOF'
#!/bin/bash
# chrun.sh: pull i run (potreban root za run; docker samo za pull)
set -e
cmd=$1 img=$2; shift 2
tarball="${img//\//_}.tar"                          # npr. redis → redis.tar
case $cmd in
  pull)
    cid=$(docker create "$img")                     # kontejner iz slike, bez
pokretanja
    docker export "$cid" -o "$tarball"              # cijeli fajl sistem kao
jedan tar
    docker rm "$cid" >/dev/null ;;
  run)
    root=$(mktemp -d "/tmp/chrun-$(tr -cd '[:alnum:]' <<<"$img").XXXXXX")
    trap 'rm -rf "$root"' EXIT                      # pri izlasku briše
privremeni korijen
    tar -xf "$tarball" -C "$root"
    "${RUNTIME:-chroot}" "$root" "$@" ;;            # ostatak argumenata je
naredba
esac
EOF
chmod +x chrun.sh
```

bash — Isječak 9 · prva slika (potreban Docker)

```
./chrun.sh pull alpine                              # jednom: skida sliku kao
alpine.tar
sudo ./chrun.sh run alpine /bin/sh -c 'cat /etc/os-release'
```

izlaz (verzija slike se može razlikovati)

```
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.24.2
PRETTY_NAME="Alpine Linux v3.24"
```

Drugi red isječka 9 ispisuje opis operativnog sistema unutar korijena izvučenog iz slike `alpine`, što potvrđuje da je pokrenut fajl sistem slike, a ne domaćina. Svaki izvršeni `run` koristi novi privremeni direktorij (`/tmp/chrun-alpine.XXXXXX`) koji se briše pri izlazu.

6.6 Primjer: Redis server i klijenti

Server i klijent pokreću se iz iste slike, svaki u vlastitom privremenom korijenu. Dok server radi, bilo koji klijent može postavljati i čitati vrijednosti.

bash — Isječak 10 · Redis iz Docker slike (potreban Docker)

```
./chrun.sh pull redis
sudo -v                                                  # unaprijed traži lozinku
sudo ./chrun.sh run redis /usr/local/bin/redis-server >/dev/null 2>&1 &   # server u pozadini
sleep 2
sudo ./chrun.sh run redis /usr/local/bin/redis-cli set hello world        # klijent 1
sudo ./chrun.sh run redis /usr/local/bin/redis-cli get hello              # klijent 2: "world"
sudo pkill redis-server                                  # kraj servera = brisanje njegovog korijena
```

izlaz

```
OK
world
```

Klijent i server komuniciraju bez mapiranja portova jer nije uveden zaseban mrežni prostor imena (*network namespace*): dijele mrežu domaćina. Izolacija fajl sistema i izolacija mreže su dakle neovisna svojstva, što vodi na sljedeće poglavlje.

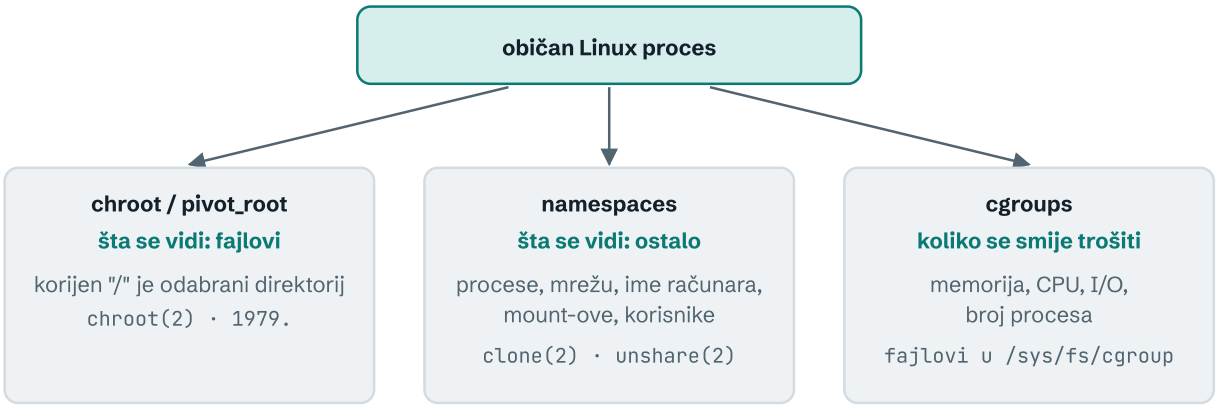
> **Kontejner je rješenje problema dinamičkog povezivanja: programu se daje cijeli fajl sistem u kojem se sve nalazi na očekivanim putanjama.**

---

7 · NADOGRADNJA

Namespaces i cgroups

Do ove tačke korišten je isključivo jedan poziv iz 1979. godine. Pravi kontejner dodaje još dva mehanizma jezgra Linuxa, koji se pojavljuju znatno kasnije, što je i razlog zašto kontejneri postaju raširen alat tek desetljećima nakon `chroot`-a.



Slika 7. Nijedan od tri mehanizma nije specifičan za kontejnere: svaki se može primijeniti na bilo koji proces. Kontejner je njihova kombinacija nad jednim procesom.

7.1 Namespaces: šta proces vidi

Namespace daje procesu zaseban pogled na globalni resurs, tako da mu se čini da ga posjeduje isključivo on. To je glavni izvor dojma da se radi o virtualnoj mašini: proces ima vlastito ime računara, vlastiti popis procesa i korisnika, iako koristi isto jezgro.

NAMESPACE	ZASTAVICA	IZOLIRA
UTS	`CLONE_NEWUTS`	ime računara
PID	`CLONE_NEWPID`	brojeve procesa (prvi proces u njemu je PID 1)
Mount	`CLONE_NEWNS`	tabelu mount-ova
Network	`CLONE_NEWNET`	mrežne interfejse, portove i rute
User	`CLONE_NEWUSER`	korisničke i grupne identifikatore
IPC, cgroup	`CLONE_NEWIPC`, `CLONE_NEWCGROUP`	IPC objekte; pogled na cgroup hijerarhiju

Namespace se uspostavlja pozivom `clone()` pri stvaranju procesa ili pozivom `unshare()` za proces koji već postoji. Isječak 11 proširuje `chrun_min` za UTS, PID i mount prostor imena. Montiranje `/proc` je potrebno da bi `ps` prikazivao samo procese novog prostora.

bash + C — Isječak 11 · chrun_ns: chroot + namespaces

```
cat > chrun_ns.c <<'EOF'
// chrun_ns.c: chroot + namespaces (UTS, PID, mount)
#define _GNU_SOURCE
#include <sched.h>
#include <stdio.h>
#include <sys/mount.h>
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc < 3) { fprintf(stderr, "upotreba: %s <root> <program> [args]\n",
argv[0]); return 1; }

    // novi hostname, novi PID prostor, novi mount prostor
    if (unshare(CLONE_NEWUTS | CLONE_NEWPID | CLONE_NEWNS) < 0) {
perror("unshare"); return 1; }

    pid_t pid = fork();            // tek dijete postaje PID 1 u novom PID
prostoru
    if (pid == 0) {
        sethostname("kontejner", 9);
        if (chroot(argv[1]) < 0 || chdir("/") < 0) { perror("chroot");
_exit(1); }
        mount("proc", "/proc", "proc", 0, NULL);   // da ps vidi samo procese
ovog prostora
        execv(argv[2], &argv[2]);
        perror("execv"); _exit(1);
    }
    waitpid(pid, NULL, 0);
    return 0;
}
EOF
gcc chrun_ns.c -o chrun_ns
sudo ./chrun_ns root3 /bin/busybox sh -c 'hostname; echo pid=$$; ps'
```

izlaz

```
kontejner
pid=1
PID   USER     COMMAND
    1 0        {ps} /bin/busybox sh -c hostname; echo pid=$$; ps
```

Proces ima PID 1, drugačije ime računara i vidi samo sebe u popisu procesa, a pri tome je običan proces domaćina. Razliku prema `chrun_min` najjasnije pokazuje sljedeći isječak,

u kojem se ime računara domaćina ne mijenja:

bash — Isječak 12 · UTS namespace u praksi

```
echo "domaćin:             $(hostname)"
sudo ./chrun_min root3 /bin/busybox hostname      # bez UTS namespace-a: ime domaćina
sudo ./chrun_ns  root3 /bin/busybox hostname      # s UTS namespace-om: "kontejner"
echo "domaćin poslije:      $(hostname)"          # domaćin ostaje nepromijenjen
```

izlaz (ime računara domaćina se razlikuje)

```
domaćin:             racunar
racunar
kontejner
domaćin poslije:      racunar
```

Novo izvršno okruženje se može primijeniti i na stvarnu sliku, tako što se u `chrun.sh` umjesto `chroot` navede `chrun_ns` kroz promjenljivu `RUNTIME`:

bash — Isječak 13 · chrun_ns nad slikom (potreban Docker, nakon isječka 9)

```
sudo RUNTIME=./chrun_ns ./chrun.sh run alpine /bin/sh -c 'hostname; echo pid=$$; ps'
```

izlaz

```
kontejner
pid=1
PID   USER     TIME  COMMAND
    1 root      0:00 ps
```

Ispis potvrđuje isti učinak kao u isječku 11: ime računara `kontejner`, `pid=1` i popis procesa u kojem je samo `ps` (PID 1), jer ljuska zamjenjuje sebe posljednjom naredbom.

7.2 cgroups: koliko proces smije trošiti

Dok *namespaces* ograničavaju šta proces vidi, *cgroups* (*control groups*) ograničavaju šta proces smije trošiti: memoriju (prekoračenje dovodi do prekida procesa od strane jezgra), CPU, ulazno-izlazne operacije i broj procesa. Sučelje je virtualni fajl sistem: jezgru se upisom u fajlove saopćava željena postavka.

bash — Isječak 14 · ograničenje broja procesa (cgroup v2)

```
cat > cg.sh <<'EOF'
#!/bin/bash
# cgroup v2: grupa "demo" smije imati najviše 5 procesa
echo +pids > /sys/fs/cgroup/cgroup.subtree_control 2>/dev/null || true   # uključi pids kontroler
mkdir -p /sys/fs/cgroup/demo
echo 5 > /sys/fs/cgroup/demo/pids.max
# podljuska ulazi u grupu i pokušava pokrenuti 10 procesa
bash -c 'echo $$ > /sys/fs/cgroup/demo/cgroup.procs; for i in $(seq 10); do sleep 3 & done; wait'
EOF
sudo bash cg.sh 2>&1 | head -3        # očekuje se: "fork: retry: Resource temporarily unavailable"
sleep 5; sudo rmdir /sys/fs/cgroup/demo
```

izlaz

```
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
```

Grupa je ograničena na pet procesa, pa jezgro odbija svaki sljedeći `fork`. Ista tehnika vrijedi za limit memorije (`memory.max`), koji Docker primjenjuje pri opciji `--memory`.

7.3 Šta bi pravo izvršno okruženje dodatno uradilo

- `pivot_root` **umjesto** `chroot`**:** iz chroot-a je moguće izaći ako proces dobije referencu izvan njega. Zato prava izvršna okruženja koriste `pivot_root`. Općenito, `chroot` se ne smije smatrati sigurnosnom granicom.
- **Slojeviti (*union*) fajl sistem:** prema slici 5, slojevi se montiraju bez raspakivanja i brisanja cijele slike pri svakom pokretanju.
- **Lijeno učitavanje (eStargz):** običan tar nema indeks, pa se za uvid u sadržaj mora u cijelosti raspakovati. „Pretraživi tar“ (*seekable tar*) ima indeks, pa kontejner može početi prije nego se slika u cijelosti preuzme: datoteke se dohvataju s registra po zahtjevu procesa. Na Kubernetesu to skraćuje pokretanje slika koje još nisu u kešu.

Šta model „proces + chroot + namespaces + cgroups“ objašnjava

PITANJE	OBJAŠNJENJE
Koliko dodatno opterećenje nosi izvršavanje u kontejneru?	Praktično nikakav: kontejner je običan proces bez posrednika.
Zašto je Docker sporiji na Macu i Windowsu?	Pozivi poput `chroot` i namespaces postoje samo u Linuxu, pa se ispod nalazi Linux virtualna mašina. Njoj je često potrebno više memorije, a dijeljenje foldera prelazi granicu virtualne mašine.
Zašto se slika sporo pokreće?	Najčešće je uzrok slika (veličina, preuzimanje, montiranje), a ne kontejner. Statički program u praznoj slici nema šta čekati.
Kada kontejner, a kada virtualna mašina?	Kontejner dijeli jezgro domaćina (brz i lagan, slabija izolacija). Virtualna mašina ima vlastito jezgro (jača izolacija, veći utrošak resursa).